

Spacepotatis skills — measured token savings

2026-05-22. A/B-tested 13 skills against a no-skill baseline. Real measurement, not a model.

The portfolio's skills-registry PDF estimates token savings using a 3× heuristic: it assumes that doing the same work without a skill would cost roughly three times as much, so saved $\approx 2 \times \text{cost-per-use} \times \text{annual uses}$. This document tests that assumption against every non-redirect Spacepotatis skill, with both arms measured in real Claude Code transcripts.

How the measurement was done

For each of the 13 skills, two Sonnet sub-agents ran in parallel in fresh sandboxed worktrees branched from Spacepotatis master at 94655a9 :

Arm A (no skill). Given the task in plain language, with explicit instruction not to read `.claude/skills/*` or any `SKILL.md`. The agent scouted the codebase cold, mirrored patterns from similar existing code, wrote the artefact.

Arm B (with skill). Given the same task, with explicit instruction to read the relevant `SKILL.md` and follow its procedure exactly.

Both arms produced real files (writes to the worktree, never committed). The token count for each arm is taken directly from the harness's per-sub-agent `usage.total_tokens`, using the same accounting convention as `/mikko-skill-usage: input_tokens + output_tokens + cache_creation_input_tokens; cache_read_input_tokens` excluded as cache hits paid for upstream. **N = 1 per skill** — single data point, calibration not benchmark.

Results

SKILL	NO SKILL (TOKENS)	WITH SKILL	SAVED	% SAVED
modular-architecture-audit	175,187	128,773	+46,414	26%
ai-codegen-smell-audit	157,348	81,119	+76,229	48%
security-audit	107,885	112,211	-4,326	-4%
content-audit	99,335	77,026	+22,309	22%
equipment	89,006	93,683	-4,677	-5%
new-perk	82,299	55,705	+26,594	32%
new-mission	78,730	49,780	+28,950	37%
new-enemy	75,309	68,353	+6,956	9%
save-roundtrip-audit	70,080	62,874	+7,206	10%
new-solar-system	65,319	59,603	+5,716	9%
new-story	47,182	33,460	+13,722	29%
balance-review	46,279	33,160	+13,119	28%

SKILL	NO SKILL (TOKENS)	WITH SKILL	SAVED	% SAVED
new-migration	24,685	20,649	+4,036	16%
Aggregate (13 skills)	1,118,644	876,396	+242,248	22%

Sorted by no-skill cost descending. Positive numbers mean the skill saved tokens vs the unstructured baseline. Negative numbers mean the skill cost more.

What the results mean

The measured net savings rate is about **22%** across these 13 skills. The PDF's heuristic implies **~67%**. The heuristic overstates savings by roughly a factor of three when averaged across this portfolio.

Most skills save real but modest tokens. The `new-*` family (new-enemy, new-mission, new-perk, new-solar-system, new-story, new-migration) plus balance-review all came in between 9% and 37%. These are scaffolding skills — the procedure tells the agent which files to touch and how. The no-skill arm has to figure that out by scouting similar existing code, and the scout cost is the savings.

Some skills save tokens by being stricter, not faster. `ai-codegen-smell-audit` saved 48% — but the no-skill arm found 11 smells while the with-skill arm found 1. The "savings" partly reflect the skill's tighter pattern definitions filtering out noise. Real efficiency win if you trust the discrimination; not a like-for-like comparison.

Two skills cost more per use. `security-audit` (−4%) and `equipment` (−5%) ran more expensive with the skill than without. Both encode *rigor* — security-audit walks the full attack surface in a prescribed order; equipment requires the full CRUD lifecycle (data, types, sprite, mission rewards, loot pools, schemas, tests). The no-skill arm has latitude to be less thorough; the with-skill arm doesn't. Their value is completeness, not token compression.

Caveats. N = 1 per skill — every number above would land differently on a re-run. Sub-agent execution isn't identical to a user driving the skill in their own session; context-loading differs. Outcome equivalence wasn't verified beyond "both arms produced something" and "tests still pass" — the smell-audit row above shows that token savings can hide a quality regression. Selection bias: the chosen tasks are representative of small create-or-audit work, not the rare end-to-end uses of orchestrator skills like the multi-phase audits.

Cost to run this entire calibration: about **2.0M tokens** across 26 sub-agent dispatches. One-shot measurement, not a routine check.

What to do with this

Three options for the skills-registry PDF, in order of smallest change to most honest:

(1) Lower the multiplier from 3× to ~1.28× — the simplest fix. The current model is $saved = (baseline - skill) \times uses$ with $baseline = 3 \times skill$. Drop baseline to 1.28× skill and the modeled savings line up with measurement at the portfolio level. One number to change.

(2) Per-skill overrides on the receipts. Set `tokens_saved_per_use` directly: zero (or slightly negative) for security-audit and equipment, ~0.25× cost for the scaffolding family, ~0.20× cost for the audits. More granular, more honest about which skills compress and which don't.

(3) Drop the savings column entirely. Present cost-per-use measurement only; stop claiming savings until $N > 1$ across multiple skills. Most honest reading of $N = 1$ data, but loses a useful framing.

The recommended path is (1) as the smallest defensible change, with a footnote on the method page citing this calibration.

Source: `docs/audits/spacepotatis-skills-calibration-2026-05-22.md` on the mikkonumminen.dev repo.

Methodology and raw numbers identical to that document — this PDF is the concise version.